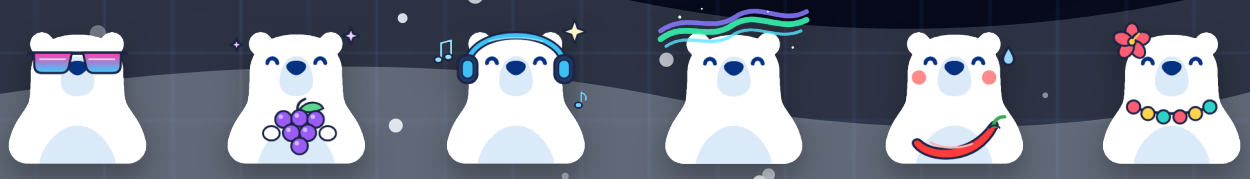
nanuq

TECHNICAL MANUAL

How Nanuq is built

Architecture with diagrams, each platform inside out and, at the end, every function in the code.





Version 1.44.6 · November 8, 2026 · English · Free software (GPL-3.0-or-later)



Contents

Part 1 explains the design; part 2 is the reference for every function.

ARCHITECTURE

1 **Overview** 6
One interface, three wrappers.

2 **The repository** 8
What lives in each folder.

3 **The page** 9
How app.js starts, navigates and reacts.

4 **Data and storage** 11
Anything about you is encrypted.

5 **Playback** 13
Where each song comes from and what happens when something fails.

6 **Library and Jellyfin** 14
The API Nanuq uses and what it builds on its own.

7 **DJ Nanuq** 15
Rule-based rounds, no AI.

8 **My devices** 16
One Nanuq plays and the others control it, through Jellyfin.

9 **The party** 17
Phones sending songs to a computer over the local network.

10 **The Android app** 19
WebView + native Java.

11 **The desktop app** 20
Electron for Linux and Windows.

12 **Security** 21
Nothing from outside is trusted.

13 **Building and releasing** 22
One command for every platform.

14 **Conventions and pitfalls** 23
What to know before changing anything.

15 **License** 24
GPLv3 or later, file by file.

DEEP LEVEL: EVERY FUNCTION

16 **web/app.js** 27
586 functions, in file order.

Base: helpers and storage 27
Server addresses 28
Jellyfin API 28
Audio: waveforms and synthesizer 30
Demo library 30
Settings and themes 31
Downloads (Android) 32
State and routing 33
Albums from tags 33
Lyrics search 33
Rendering helpers 34
Views 34
DJ Nanuq 34
Your year and Your week (video) 37
Song requests (SomeDL) 38
Player 39
Lyrics 41
Queue 42
Toasts, menus and dialogs 42
My devices 42
Party (watch party) 44
Global actions 46
Login 46

17 **Rest of the page** 48
Languages and service worker.

web/i18n.js 48

web/sw.js 48

18 **Android** 49
Native Java and the JavaScript bridge.

android/app/src/main/java/com/nanuq/app/Discovery.java 49
android/app/src/main/java/com/nanuq/app/Downloads.java 49
android/app/src/main/java/com/nanuq/app/LocalServer.java 50
android/app/src/main/java/com/nanuq/app/MainActivity.java 51
android/app/src/main/java/com/nanuq/app/MediaCenter.java 52
android/app/src/main/java/com/nanuq/app/NativeHttp.java 53
android/app/src/main/java/com/nanuq/app/PlaybackService.java 53
android/app/src/main/java/com/nanuq/app/Prefetch.java 53
android/app/src/main/java/com/nanuq/app/SecureStore.java 54
android/bridge/android-bridge.js 55
android/build-apk.sh 55

19 **Desktop** 56
Electron and its packaging.

linux/main.js 56
linux/preload.js 58
linux/prepare-web.js 58
linux/collect.js 58
linux/flatpak/build-flatpak.sh 58

20 **Build** 59
The script that builds everything.

scripts/build.sh 59

PART 1

Architecture

How Nanuq is built: a single web interface, the apps that wrap it and how it talks to Jellyfin, to your other devices and to the party.

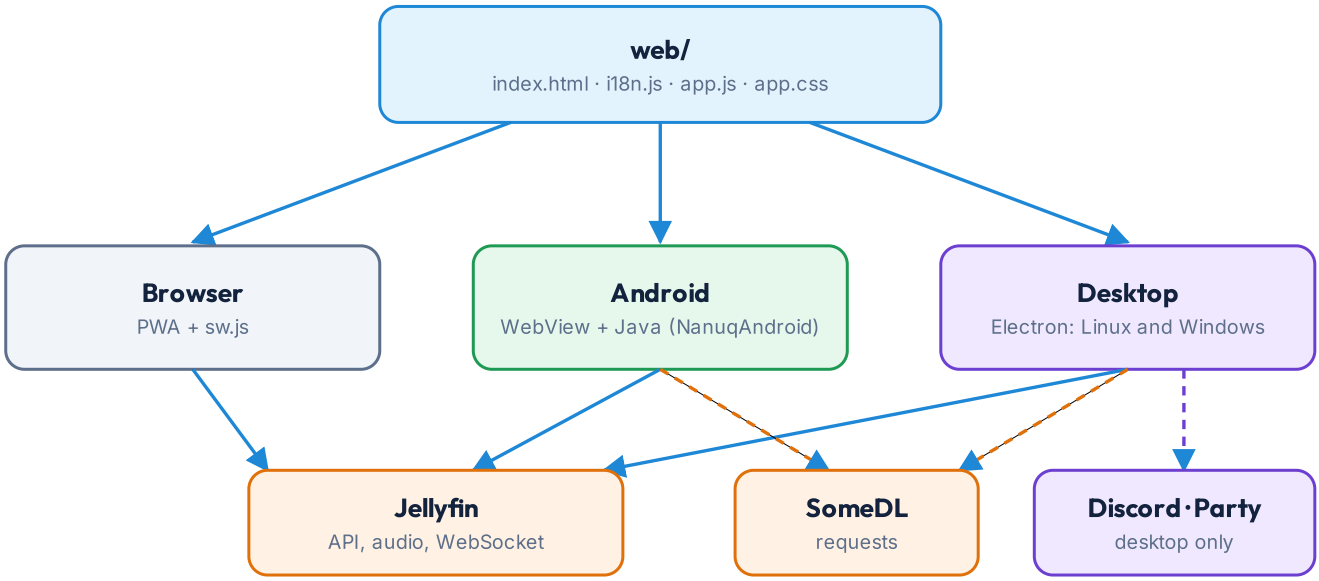


01 Overview

One interface, three wrappers.



All of Nanuq's interface lives in `web/`: four files, with no build step and no dependencies. The Android and desktop apps have no interface of their own: they copy that same folder at build time and add, through a “bridge”, what a web page can't do on its own.



Every platform serves the same page; dashed arrows are what only native code does (SomeDL without CORS, Discord, the party room).

Why this way

- **One change reaches every platform**: the interface is only edited in `web/`; Android and Linux copy it at build time (`syncWeb` in Gradle, `prepare-web.js` in Electron).
- **Secure context without HTTPS**: Android serves the page at `http://localhost` and Electron at `http://127.0.0.1:47863`. That gives `crypto.randomUUID` and storage (the service worker is only used in the browser), while still talking to an `http://` Jellyfin on the local network without mixed-content blocks.
- **No third-party services**: Nanuq only talks to your Jellyfin, your SomeDL and (if you enable it) the Discord app on your computer. The only external requests are the Google Fonts.

Ports

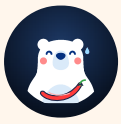
Port	Who	What for
TCP 47863	Desktop (127.0.0.1)	Serves the page and, with the party open, the room's audio (`/fiesta/<token>/…`). Fixed port because storage depends on the origin.
TCP 47864	Desktop (local network)	Party WebSocket (`/fiesta`) and covers for guests. Only with the room open and only private addresses.
UDP 47865	Desktop	Answers “Who is NanuqParty?” so phones find the room (≤ 30 answers/s).
UDP 7359	Jellyfin	Official server discovery (“Who is JellyfinServer?”), from Android and desktop.
TCP random	Android (127.0.0.1)	`LocalServer` : downloads, covers and prefetched songs, with a token in the path.
8096 / 8920	Jellyfin	Ports assumed if you type an IP or short name without a port.
5000	SomeDL	Port assumed for SomeDL without a port.

02 The repository

What lives in each folder.



Path	Contents
`web/index.html`	Structure, strict CSP and SVG `<defs>`: `#i-*` icons and the vector bear (`#djb-*`).
`web/i18n.js`	Languages: `LANGS`, `LANG`, `LOC`, the `EN` dictionary (key = Spanish text) and `tx()`. Loaded before app.js.
`web/app.js`	All the logic (5,955 lines). `const VERSION` is the source of truth for the version.
`web/app.css`	All styling, with per-theme variables (18 themes).
`web/sw.js`	Service worker, browser only: caches the app (network first for code), never music or API. Android and desktop don't register it.
`android/`	WebView + Java (`com.nanuq.app`), the `bridge/android-bridge.js` bridge and `build-apk.sh`.
`linux/`	Electron: `main.js`, `preload.js`, `prepare-web.js`, `collect.js` and `flatpak/`. It also builds the Windows installer.
`scripts/build.sh`	Bumps the version and builds everything into `dist/<version>/`.
`docs/`	README images and these manuals (`docs/manuals/src`: `node docs/manuals/src/build.js`).



Heads-up

`dist/`, `keystore.properties` and `*.jks` files never go into the repository. The APK signing key lives outside (`~/.android/nanuq-release.jks`): without it, updates can't be published.

03 The page

How app.js starts, navigates and reacts.



Startup

1. On load, `i18n.js` picks the language and `txStatic()` translates the fixed HTML texts.
2. `SEC` detects the native encrypted store and `secMigrate()` moves there whatever older versions left in plain text.
3. The boot block draws the sidebar and tabs, applies the theme and starts downloads (`DL.init`).
4. `Vault.init()` retries for a few seconds if the Keystore or keyring isn't ready yet; then `Vault.load()` reads the session.
5. With a session: `startApp(new Jellyfin(sess))` restores the queue and position (`QPOS`), restores the DJ and opens Home. Without one: `showLogin()`.

Navigation

`S.stack` is the screen stack. `go(name, params, opts)` pushes a screen (the `TOP` ones clear the stack) and calls `render()`, which runs `V[name](box, params)`. Each view is an async function that fills its box; if it leaves a `_cleanup`, it's called on exit. `back()` first closes whatever is on top (menu, lyrics editor, player) and then goes back. With the party open (`party-host` / `in-party`) only the party screen exists.

Events

The CSP forbids inline scripts, so there's no `onclick=`: every button has `data-act="…"` and a single global click handler decides what to do (“Global actions” section). Images that must disappear when they fail carry `data-rm`.

Languages

Every visible text is written in Spanish inside `tx('…')`; `i18n.js` holds the translation keyed by the Spanish text. With values: `tx('… {0} …', [x])`. Switching language saves the position and reloads the page, so texts built at load time are translated too.

Sections of app.js

Section	From line
Base: helpers and storage	1
Server addresses	122

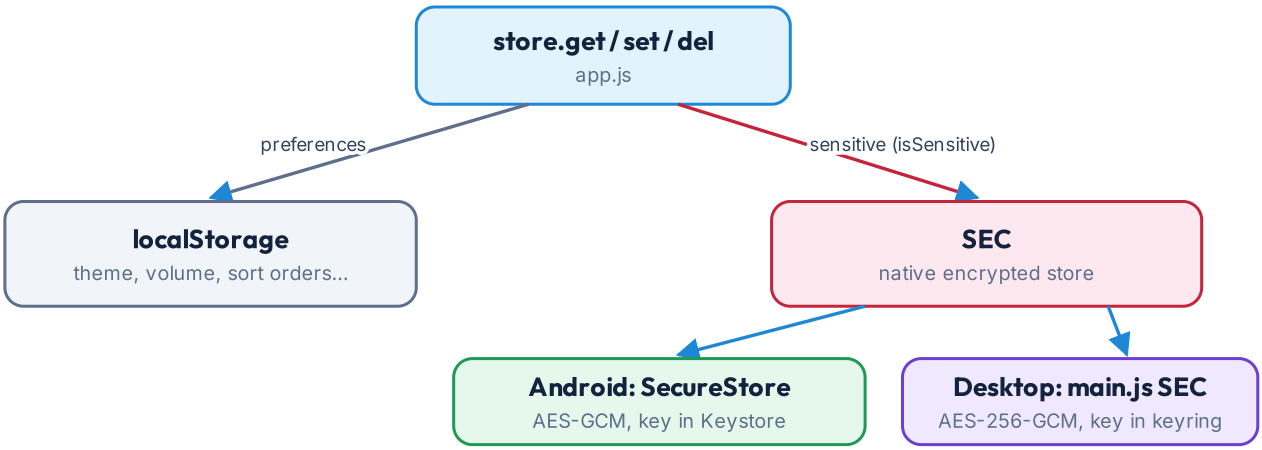
Section	From line
Jellyfin API	177
Audio: waveforms and synthesizer	490
Demo library	551
Settings and themes	749
Downloads (Android)	794
State and routing	947
Albums from tags	997
Lyrics search	1102
Rendering helpers	1200
Views	1230
DJ Nanuq	1297
Your year and Your week (video)	2221
Song requests (SomeDL)	2937
Player	3228
Lyrics	3716
Queue	4062
Toasts, menus and dialogs	4133
My devices	4144
Party (watch party)	4842
Global actions	5586
Login	5746
Boot	5926

04 Data and storage

Anything about you is encrypted.



`store` is the only way to storage. Interface preferences (theme, volume, sort orders) go to `localStorage` with the `nanuq.` prefix; anything that says something about you goes to the platform's encrypted store. Since preferences aren't encrypted, they're validated when read (right after `Settings`): an altered or outdated value goes back to the default.



In the browser there is no secure store: everything stays in localStorage (Settings says so).

Key	What it holds
session	Jellyfin server, user and token.
queue · queuePos	Your queue (up to 3000) and the second you were at (queuePos isn't sensitive: just a random id and seconds).
libIdx · lyrIdx	Indexes of your library (albums from tags) and of your lyrics.
dj · djHist	Current DJ session and history of what it played.
plays · mp3ids	Plays per song on this device and songs that go as MP3.
somedl · somedlReqs · somedlLast	SomeDL server and your requests.
servers · lastServer · device · party	Recent servers, device id and the current party (phone).
lrc.<id>	Lyrics saved only on this device.

Store states

`SEC.kind` can be the store's name (works), `locked` (encrypted data that won't open right now: retried), `lost` (the key is gone: “Start over” is offered, which sets the data aside without deleting it) or `null` (no store). While it isn't open, new data lives only in memory (`SEC.mem`): it's **never** written in plain text.

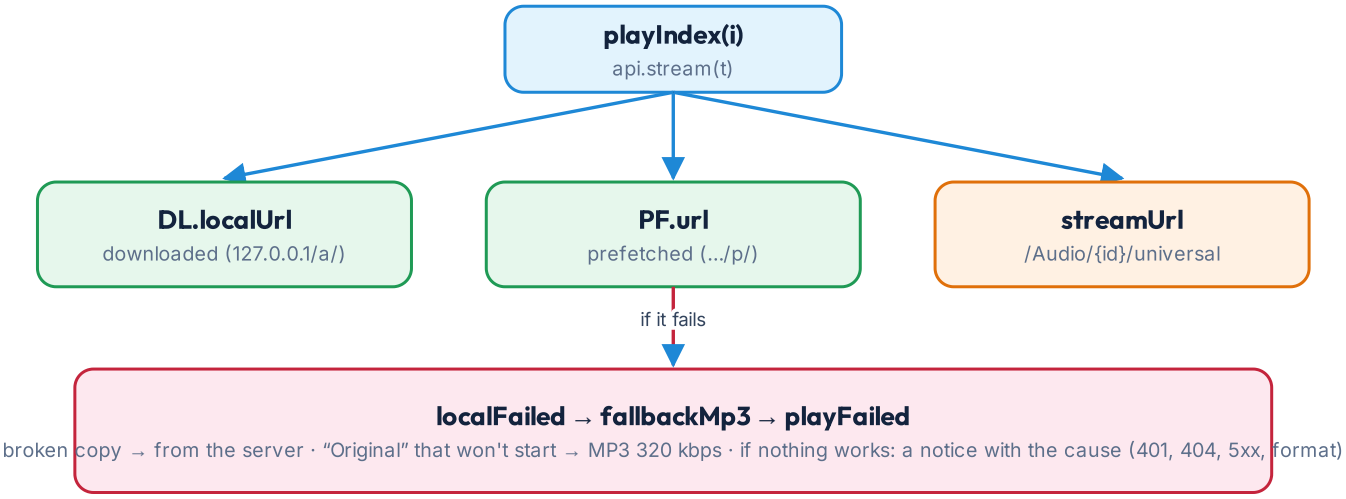
05

Playback

Where each song comes from and what happens when something fails.



`P` holds the queue, the index and the single `<audio>`. `playIndex(i)` picks the source with `api.stream()`: first the downloaded copy, then the prefetched one and, if neither exists, Jellyfin streaming.



On Android, the first two sources are served by the phone itself; on Android and desktop the page never loads the whole audio into memory.

Details that matter

- **Seeking in live transcodes**: if Jellyfin transcodes, the file can't be seeked; `seekTo` requests the audio again with `StartTimeTicks` and remembers the offset (`P.off`). While dragging, the request goes out on release.
- **Stalls**: `armStallWatch` warns if it doesn't start within 10 s; `stallCheck` resumes from the same second if the network dropped mid-song; `netBack` does it when the network returns.
- **Fades**: `pauseSoft` / `playSoft`; the restored volume is always yours (`userVol`), never the one of that instant.
- **Foreign pause**: if another app took the audio output (`P.extPause`), resuming reloads the song so it doesn't stay silent.
- **System controls**: `navigator.mediaSession`; on Android the native bridge replaces it. On desktop a silent audio (`rcKeep` / `sessionHold`) keeps the controls while loading or while you control another device.
- **Favorites**: Jellyfin 12 overwrites the favorite of the playing song with every playback report; `FAVHOLD` saves it again after each report.

06

Library and Jellyfin

The API Nanuq uses and what it builds on its own.



Jellyfin route	What for
/System/Info/Public · /Users/AuthenticateByName	Recognize the server and log in.
/Items · /Artists/AlbumArtists · /MusicGenres · /Playlists	Library lists (with several fallback routes depending on the version).
/Audio/{id}/universal	The audio (streaming, downloads, prefetch, party).
/Audio/{id}/Lyrics · .../RemoteSearch/Lyrics	Read, upload and search lyrics (10.9+).
/UserFavoriteItems/{id}	Favorites (and the pre-10.9 route as fallback).
/Items/{id}/InstantMix	Song radio.
/Sessions · /Sessions/{id}/Playing · /socket	My devices: sessions, commands and live events.
/Sessions/Playing(/Progress|/Stopped)	Playback reports (what the server sees).
/DisplayPreferences/nanuq-dj	DJ state shared between your devices.
/user_usage_stats/...	Playback Reporting plugin for Your week / Your year (if present).

Albums from tags (LIB)

Jellyfin builds albums by folder; Nanuq builds them from each song's “Album” + “Album artist” tags (`albumKeyOf`), joins “(Disc 2)” with its album and removes duplicates. The index of all your songs is fetched 1000 at a time (three pages at once), stored encrypted and refreshed every 10 minutes. These album ids start with `m` (`albumGid`).

Lyrics search (LYRX)

Jellyfin doesn't search inside lyrics: Nanuq indexes the songs with `HasLyrics` (3 at a time, up to 8000) plus local lyrics, normalized without accents or punctuation (`lyrNorm`). Searches need 4 characters or more.

07

DJ Nanuq

Rule-based rounds, no AI.



`DJ.next()` shuffles round types by weight, avoids repeating recent ones and tries until it finds one that **explores** (songs that didn't play in recent weeks). Each song carries its round in the queue id: `dj—<session>-<round>-<random>`.

Round	Weight	What it plays
genre	6	A genre from your library (≥ 3 songs in a 400-song sample), not repeating recent ones.
decade	3.5	A real decade from your library; past ones weigh ×1.5.
discover	3	Never played, first in your favorite genres.
forgotten	3	More than 45 days without playing.
mood	1	By time of day (calm morning, energetic afternoon…).
fresh · fav · recent · top	.8 · .6 · .15 · .15	Newly added, favorites, recent days and most played (with cooldown).

`DJ.rank()` decides how “worn out” a song is: anything from the last 3 days or from recent DJ sessions barely gets in; forgotten songs go first. When music moves to another device, `DJSHARE` leaves the state in your Jellyfin and the other one adopts it (`DJ.adopt`); whatever is read is cleaned with `djClean`.

08 My devices

One Nanuq plays and the others control it, through Jellyfin.





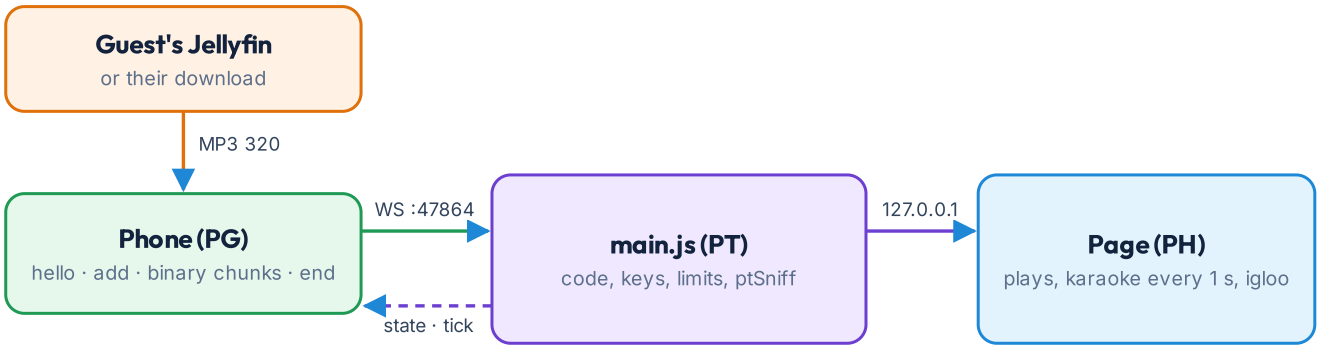
No port is opened on the devices: everything goes through your Jellyfin, with your session.

- **Who shows up**: sessions with client “Nanuq”, of your same user, active within 15 minutes by the *server's* clock (skewed clocks don't matter).
- **Same network**: same /24 in IPv4 or /64 in IPv6 (or both private), using the address the server sees.
- **Accepted commands**: `Play` and `GeneralCommand` only if your user sent them (data Jellyfin fills in); `Playstate` is filtered by Jellyfin permissions.
- **While controlling**, your local queue is set aside (`RC.localQ`) and comes back when you disconnect; the position is estimated from the last report (`RC.anchor`).
- On Android, with the screen locked, the native app calls `__nanuqTick` every 20 s to keep the WebSocket alive.

09 The party

Phones sending songs to a computer over the local network.





UDP :47865 "Who is NanuqParty?" → room name and port

The phone fetches the song from ITS Jellyfin and hands it to the computer in 256 KB chunks, without holding it whole in memory.

Message	Direction	What it is
hello {code|key, name, theme}	phone → PC	Join with the 6-digit code or come back with its key.
welcome {you, key, room, chunk}	PC → phone	Joined: its key to come back without the code (2 h).
add · [id + bytes] · end · remove	phone → PC	Send a song (data, lyrics, cover) and its bytes.
addOk · addErr · ready	PC → phone	Answer to the request and notice that it's ready.
state · tick	PC → phones	List, guests and what's playing; second and karaoke line.
bye {why} · leave · ping	both	Leave, be removed or say it's still there.

Limits and defenses

`PT_MAX` in `main.js`: `{ guests: 12, conns: 24, file: 80e6, disk: 800e6, list: 60, perGuest: 10, frame: 1 << 20, text: 450e3, lyrics: 200e3, cover: 150e3 }`. Private addresses only (`ptLocal`), ≤ 4 not-yet-identified connections per address, 5 wrong codes per minute per address and, after 30 failures overall, a new code and one minute closed. JSON is read with `jparse` and texts with `ptStr`; a message that breaks something only drops that connection.

Songs on disk, encrypted

Songs go to a folder on disk (`ptBase()`: `~/.cache/nanuq/nanuq-fiesta-*` on Linux; in the Flatpak, inside `~/.var/app/com.nanuq.app/cache`), not to the temp folder, which is RAM there. A song is refused if less

than 1 GB is free. Each song is stored **encrypted and authenticated** with AES-256-GCM in 64 KB blocks (`PT_CH`): the key is random and lives only in the room's memory (`PT.fkey`), and the nonce is 8 random bytes per song plus the block number (`ptNonce`). `ptSeal` encrypts on arrival; `ptServe` checks the exact size (`ptEncSize`) and `ptOpenRange` decrypts only the blocks of the requested range. If another app changes a byte, truncates the file or swaps blocks, that block isn't served and the song leaves the list.

10 The Android app

WebView + native Java.

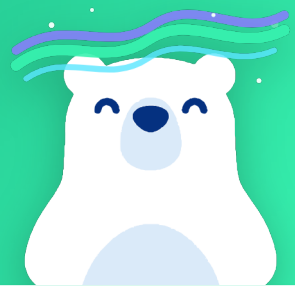


Class	Responsibility
MainActivity	WebView (assets at `http://localhost`), the `NanuqAndroid` bridge, 20 s heartbeat, recreate the WebView if it dies, external links http(s) only.
MediaCenter · PlaybackService	MediaSession and notification; foreground service with CPU and Wi-Fi locks while playing; released after 10 min paused.
Downloads	One-at-a-time downloads, Wi-Fi only by default, limit with replacement of automatic ones; encrypted index.
Prefetch	What's next in the queue, in the cache (≤ 400 MB per file, 1 GB total, 300 MB free).
LocalServer	HTTP on 127.0.0.1 with a token: `a/` audio, `c/` cover, `p/` prefetch; Range; 10 s per request, ≤ 100 headers, 32 threads.
SecureStore	AES-GCM store with a key in Android Keystore; locked / lost states.
NativeHttp · Discovery	HTTP without CORS only for SomeDL routes; UDP discovery for Jellyfin and parties.

The `android-bridge.js` bridge is injected into `index.html` at build time and replaces `navigator.mediaSession` with one that talks to `MediaCenter`. Only Jellyfin-shaped URLs are downloaded (`Downloads.okUrl`) and redirects only within the same server (`openSameHost`).

11 The desktop app

Electron for Linux and Windows.



IPC channel	What it does
nanuq:discover	Finds Jellyfin servers over UDP.
nanuq:http	HTTP without CORS, SomeDL routes only, no redirects, ≤ 4 MB.
nanuq:sec (sync) · nanuq:secreset	Encrypted store: random AES-256 key saved with `safeStorage` (keyring / DPAPI).
nanuq:discord	Rich Presence over the `discord-ipc-N` socket (only sockets you own), with a minimal retry and status for Settings.
nanuq:party · nanuq:party-ev	Open/close the room and run it (jump, order, played, now, tick, lyrics).

- Every IPC channel checks `fromApp(e)`: only the page from our origin can call it.
- Window with `contextIsolation`, `sandbox` and no Node; permissions only for notifications, fullscreen and clipboard.
- Electron fuses: no `runAsNode` or `NODE_OPTIONS`, asar integrity and load only from the asar.
- **Code integrity:** no HTTP disk cache (`disable-http-cache`; the old `Cache` folder is deleted at startup) and, before the page opens, the service worker, Cache Storage and V8 code cache are cleared (also when reloading with F5). `window.json` only accepts sensible numbers.
- Local server: GET/HEAD only, `Host` 127.0.0.1/localhost (against DNS rebinding), paths ≤ 512 and no null bytes.

12 Security

Nothing from outside is trusted.



- **CSP** without inline scripts, without `eval`, iframes or `<base>`; only `style` attributes.
- **Outside JSON** through `jparse` (no `toString` / `valueOf` / `__proto__`) and texts through `str()`: never `String(x)` on foreign data.
- **HTML**: every value goes through `esc()`; ids through `sid()` (letters, digits and dashes only) before going into a path.
- **Lyrics** stripped of control characters and tags (`cleanLyricsText`), ≤ 200,000 characters.
- **Native bridges** with route allowlists and size limits; no following redirects to another server.
- **Personal data** encrypted on disk; Android backup disabled (`allowBackup=false` and extraction rules).
- **Integrity on disk**: nothing stored is used without checking. What's encrypted (personal data, downloads index, party songs) uses AES-GCM, which detects any change; unencrypted preferences are validated when read; on Android everything lives in the app's private storage.



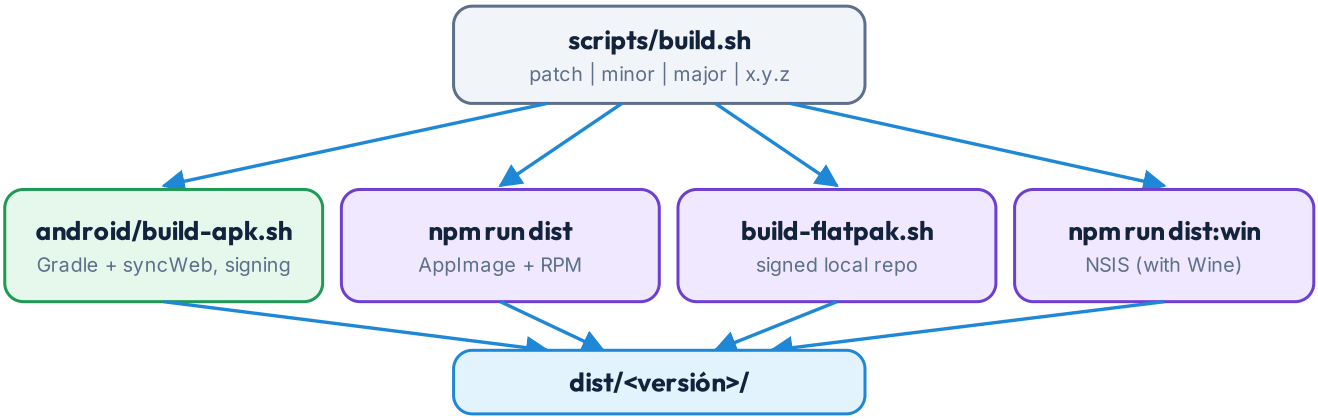
Nanuq's tip

The full detail (what is protected and known limits) is in `SECURITY.md`.

13 Building and releasing

One command for every platform.





The version lives in `web/app.js` (VERSION) and `web/sw.js` (CACHE); the APK, package.json and Flatpak take it from there.

- **Versioning**: new features bump the minor number; fixes, the last one. The APK `versionCode` is M·10000 + m·100 + p.
- **Android** needs JDK 21, the SDK and `android/keystore.properties`. Always signed with the same key.
- **Linux**: the RPM uses the system `fpm` (`USE_SYSTEM_FPM=true`). The Flatpak is built from `linux-unpacked`.
- **Checks**: `node --check web/app.js` and, for UI, Chromium in demo mode.

14 Conventions and pitfalls

What to know before changing anything.



Rule	Why
UI changes only in `web/`	Android and Linux copy it at build time.
Every visible text in `tx('…')` + its `EN` translation	Otherwise it shows in Spanish in English mode. One full sentence per `tx`, no gluing fragments.
No `onclick=`, `<script>` or `<style>` from JS	The CSP blocks them silently.
Unique ids across the whole app	A duplicated id hid the Search box (1.30.2).
Unique `view-transition-name`	If repeated, the transition aborts with no visible error.
Endless animations only in bursts (`nqBurst`)	At 120 Hz each one costs ≈ 1 phone core.
Respect `prefers-reduced-motion`	Accessibility.
After writing to Jellyfin, show the local copy and confirm later	An immediate read may return stale data (lyrics, playlists, favorites).
`.ig-crowd` without `z-index`	It would create its own layer and everyone would end up behind the DJ.

15 License

GPLv3 or later, file by file.



Nanuq is free software under the **GNU General Public License, version 3 or (at the recipient's option) any later version**: `GPL-3.0-or-later`. The full text is in `LICENSE`, at the root of the repository.

What it means for whoever modifies it

- You can use, study, copy and modify the code for any purpose.
- If you **distribute** Nanuq or a modified version (an APK, a package, the PWA on a server others download from), it must go under the same license and with its source code (or an offer to get it), keeping the copyright and license notices.
- A modified version must say it is one. For Android, use another `applicationId` and your own signing key: only the author signs the official APK (see “Building and releasing”).
- It comes with **no warranty**.

How the code is marked

Every source file starts with two lines in the **SPDX** format (the one used by license tools and the REUSE specification):

```
SPDX-FileCopyrightText: 2026 Fleyrer y colaboradores de Nanuq
SPDX-License-Identifier: GPL-3.0-or-later
```

They go as a comment in each file's language (`//` , `/* */` , `<!-- -->` or `#`), after the `#!` in scripts and after the `<!DOCTYPE>` in HTML. When you create a new file, add the same two lines.

Where else it shows up

Place	How
`LICENSE`	Official GPLv3 text (GitHub detects it and shows it on the repository).
README.md · README.en.md	Line at the top, “License” section and a link in the menu.
App settings	“Nanuq x.y.z” row: the license and a link to `LICENSE` (opens in the browser).
linux/package.json	`"license": "GPL-3.0-or-later"`
Flatpak (metainfo)	`<project_license>GPL-3.0-or-later</project_license>`
These manuals	Cover and this chapter.

What isn't Nanuq's

Component	License	Where
Electron	MIT	Inside the desktop apps (Linux and Windows).
Inter · Outfit	SIL Open Font License	Fonts, loaded from Google Fonts (not in the repository).
Gradle wrapper	Apache 2.0	`android/gradlew`
Jellyfin	GPL-2.0	The server: a separate project Nanuq talks to through its API. Nanuq is not affiliated with it.

PART 2

Deep level: every function

All 846 functions and methods in the code, file by file, with the line where they start and what they do.



16 web/app.js

586 functions, in file order.



web/app.js 5,955 lines

Base: helpers and storage from line 1 · 25

Function	Line	What it does
$	5	Shortcut for document.querySelector (optionally inside another element).
$$	6	querySelectorAll shortcut that returns an array.
jparse	10	Parses outside JSON without toString/valueOf/__proto__.
str	11	Converts any value to text without ever throwing.
esc	12	Escapes text to put it into HTML (& < > " ').
sid	16	Cleans a server id: only letters, numbers and dashes.
num	17	Converts to a number; 0 if it isn't finite.
numOrNull	18	Converts to a number or returns null if empty or invalid.
ic	30	HTML for an SVG icon from the <defs> (#i-name).
fmt	31	Formats seconds as m:ss or h:mm:ss.
isSensitive	38	Does this stored key hold personal data (goes to the encrypted store)?
SEC.ok	46	Is the native encrypted store open and usable?
SEC.shut	47	Is the store locked or lost (nothing is written in plain text)?
store.get	55	Reads a stored value (encrypted if sensitive, otherwise localStorage) and migrates old plain copies.
store.set	67	Saves a value; sensitive data goes to the encrypted store or, if closed, only to memory.
store.del	78	Deletes a stored value from the store and localStorage.
secMigrate	81	Moves into the encrypted store what older versions left in plain text.
Vault.kind	95	Name of the encrypted store in use (or null).
Vault.init	98	Cold start: retries opening the store for a few seconds before giving up.
Vault.save	101	Saves the Jellyfin session.
Vault.load	102	Reads the saved session (or flags that the store is locked).

Function	Line	What it does
`Vault.reset`	109	“Start over”: sets unreadable data aside and creates a new store.
`uid`	117	Random id (UUID without dashes).
`deviceId`	118	Persistent id of this device for Jellyfin.
`deviceName`	119	System name (Android, Linux, Windows…) announced to Jellyfin.

Server addresses from line 122 · 1

Function	Line	What it does
`parseServer`	124	Parses what you typed as a server (IPv4, IPv6, name, port and path) and builds the base URL.

Jellyfin API from line 177 · 58

Function	Line	What it does
`Jellyfin.constructor`	180	Creates the Jellyfin client from the saved session.
`Jellyfin.authHeader`	181	Builds the MediaBrowser authorization header with device, version and token.
`Jellyfin.req`	185	API request with a timeout and no cache; detects 401 and reads JSON with jparse.
`Jellyfin.alt`	203	Tries several API routes until one doesn't return 404.
`Jellyfin.probe`	204	Checks that an address is a Jellyfin (/System/Info/Public).
`Jellyfin.login`	212	Logs in with username and password and stores the token.
`Jellyfin.cfg`	216	Session data to save.
`Jellyfin.map`	217	Converts a Jellyfin item to Nanuq's format, cleaning ids, numbers and texts.
`Jellyfin.items`	236	Generic /Items query with the fields the app uses.
`Jellyfin.albums`	240	Paginated, sorted Jellyfin albums.
`Jellyfin.item`	244	One item by id (with a fallback route for old servers).
`Jellyfin.album`	245	A Jellyfin album and its tracks.
`Jellyfin.artists`	249	Album artists, paginated.
`Jellyfin.artistFilters`	254	Possible filters to find an artist's items (by several ids or by name).
`Jellyfin.firstNonEmpty`	260	Tries filters in order and returns the first non-empty result.
`Jellyfin.artist`	264	An artist and their albums (built from their songs if needed).
`Jellyfin.artistTracks`	279	All of an artist's songs.
`Jellyfin.genres`	287	Music genres trying several routes; otherwise derived from albums.
`Jellyfin.genreParam`	306	Query parameters for one or more genres (by id or by name).

Function	Line	What it does
`Jellyfin.genre`	311	A genre's albums.
`Jellyfin.genreTracks`	318	A genre's songs (from the songs' tags or their albums).
`Jellyfin.playlists`	327	Your audio playlists.
`Jellyfin.playlist`	328	A playlist and its songs.
`Jellyfin.views`	332	The user's music libraries (root of Folders).
`Jellyfin.folder`	336	A folder's contents.
`Jellyfin.search`	337	Search for artists, albums and songs.
`Jellyfin.img`	344	Cover URL (local if it's downloaded on the phone).
`Jellyfin.streamUrl`	349	Universal streaming URL with quality, containers and session.
`Jellyfin.stream`	354	URL to play: download, ahead-of-time cache or streaming.
`Jellyfin.dlUrl`	357	URL to download a song to the phone.
`Jellyfin.peaks`	363	Downloads a light copy and computes the real waveform.
`Jellyfin.tracks`	368	Paginated, sorted library songs.
`Jellyfin.lyrics`	372	A song's lyrics (Jellyfin 10.9+), normalized.
`Jellyfin.mix`	377	Songs for a mix (order, filters, genres, years).
`Jellyfin.uploadLyrics`	384	Uploads lyrics (.lrc or .txt) to Jellyfin.
`Jellyfin.searchLyrics`	391	Searches for lyrics in Jellyfin's providers (LrcLib).
`Jellyfin.pickLyrics`	395	Picks one of the found lyrics to save it.
`Jellyfin.setFavorite`	397	Marks or unmarks a favorite.
`Jellyfin.inFavorites`	403	Is it in Jellyfin's favorites section?
`Jellyfin.isFavorite`	404	Is this song a favorite according to its user data?
`Jellyfin.favPlaylist`	411	Finds the “Favorited Songs Playlist” plugin's playlist.
`Jellyfin.favorites`	418	Your favorites (IsFavorite filter), paginated and sorted.
`Jellyfin.setFavoriteLegacy`	436	Old route to save favorites (second attempt).
`Jellyfin.instantMix`	438	Jellyfin instant mix from a song (radio).
`Jellyfin.addToPlaylist`	443	Adds songs to a playlist.
`Jellyfin.refreshLibrary`	445	Asks Jellyfin to rescan the libraries (administrators).
`Jellyfin.savePlaylist`	446	Creates a new playlist with songs.
`Jellyfin.report`	447	Tells Jellyfin a song started, progressed or stopped (and reasserts favorites).
`Jellyfin.logout`	453	Logs out on the server (revokes the token).
`Jellyfin.wsUrl`	456	Jellyfin WebSocket URL.
`Jellyfin.postCapabilities`	457	Announces that this Nanuq accepts remote-control commands.

Function	Line	What it does
`Jellyfin.sessions`	459	Active sessions (for My devices).
`Jellyfin.sessionCmd`	460	Sends a playback command to another session.
`Jellyfin.sessionPlay`	461	Sends a queue to play on another session.
`Jellyfin.sessionGeneral`	462	Sends a general command (repeat, shuffle) to another session.
`Jellyfin.itemsByIds`	463	Requests several items by id, 100 at a time.
`netError`	469	Clear message when the server can't be reached.
`normLyrics`	478	Normalizes Jellyfin lyrics (synced or plain).

Audio: waveforms and synthesizer from line 490 · 6

Function	Line	What it does
`actx`	493	Shared AudioContext (created on first use).
`decode`	494	Decodes audio into an AudioBuffer.
`peaksFromBuffer`	495	Computes waveform peaks from an AudioBuffer.
`rng`	502	Seeded pseudo-random generator.
`wav`	503	Converts an AudioBuffer into a WAV file.
`synth`	510	Synthesizes a test song for demo mode.

Demo library from line 551 · 43

Function	Line	What it does
`Demo.constructor`	554	Creates the demo library (artists, albums, songs and playlists).
`Demo.cfg`	589	Demo-mode session.
`Demo.wait`	590	Simulates network latency.
`Demo.albums`	591	Demo version of Jellyfin.albums.
`Demo.album`	596	Demo version of Jellyfin.album.
`Demo.artists`	597	Demo version of Jellyfin.artists.
`Demo.artistBy`	599	Finds a demo artist by id or name.
`Demo.artist`	600	Demo version of Jellyfin.artist.
`Demo.artistTracks`	601	Demo version of Jellyfin.artistTracks.
`Demo.genres`	602	Demo version of Jellyfin.genres.
`Demo.genreTracks`	603	Demo version of Jellyfin.genreTracks.
`Demo.genre`	604	Demo version of Jellyfin.genre.
`Demo.playlists`	605	Demo version of Jellyfin.playlists.
`Demo.playlist`	606	Demo version of Jellyfin.playlist.
`Demo.views`	607	Demo version of Jellyfin.views.

Function	Line	What it does
`Demo.folder`	608	Demo version of Jellyfin.folder.
`Demo.search`	613	Demo version of Jellyfin.search.
`Demo.img`	617	Draws a generated SVG cover for demo mode.
`Demo.buf`	631	Synthesized audio for a demo song (cached).
`Demo.stream`	632	Blob URL of a demo song's WAV.
`Demo.peaks`	633	Waveform of a demo song.
`Demo.tracks`	634	Demo version of Jellyfin.tracks.
`Demo.mix`	640	Demo version of Jellyfin.mix.
`Demo.lyrics`	650	Made-up synced lyrics for a demo song.
`Demo.setFavorite`	662	Demo version of Jellyfin.setFavorite.
`Demo.isFavorite`	663	Demo version of Jellyfin.isFavorite.
`Demo.inFavorites`	664	Demo version of Jellyfin.inFavorites.
`Demo.favorites`	665	Demo version of Jellyfin.favorites.
`Demo.instantMix`	666	Demo version of Jellyfin.instantMix.
`Demo.addToPlaylist`	667	Demo version of Jellyfin.addToPlaylist.
`Demo.savePlaylist`	668	Demo version of Jellyfin.savePlaylist.
`Demo.report`	669	Does nothing (demo doesn't report plays).
`Demo.logout`	670	Does nothing (no server).
`logoInner`	675	HTML of the logo bear with the theme's accessory.
`bearHTML`	691	HTML of the animatable Nanuq (logo + expressive face).
`bearState`	693	Current bear state (dancing, happy, sad) for new bears.
`nqBurst`	699	Turns on an animation burst for a few seconds (battery saving).
`nqBurstEnd`	707	Ends the burst: animations pause and bears stay happy.
`bearDance`	712	Makes every Nanuq dance for a while.
`bearInit`	717	Puts Nanuq in the sidebar logos.
`bearMood`	722	Changes the bears' mood when music plays or pauses.
`showSplash`	733	Welcome screen with the theme's Nanuq.
`refreshBear`	744	Changes the bears' accessory when the theme changes.

Settings and themes from line 749 · 1

Function	Line	What it does
`Settings.set`	776	Changes a setting and saves it.

Downloads (Android) from line 794 · 29

Function	Line	What it does
DL.init	800	Loads the phone's downloads from the native app.
DL.indexCovers	808	Index of downloaded covers by image id.
DL.coverUrl	812	Local URL of a downloaded cover.
DL.backfillCovers	814	Fills in covers and details of old or recovered downloads.
DL.config	821	Sends Android the maximum songs and whether Wi-Fi only.
DL.usable	822	Are downloads available on this device (Android with Jellyfin)?
DL.get	823	A song's download, if any.
DL.localUrl	824	Local URL of a downloaded, ready song.
DL.isLocal	825	Is this URL from the phone's local server?
DL.add	827	Requests downloading a song (manual or automatic).
DL.remove	832	Removes a song from downloads.
DL.removeAll	833	Deletes all downloads (or only automatic ones).
DL.list	834	List of downloads.
DL.bytes	835	Total space used by downloads.
DL.net	836	Current network type according to Android (wifi, cellular, none).
__nanuqDl	838	Receives a download's changes from Android and updates the view.
Plays.count	853	Times a song played on this phone.
Plays.add	854	Counts a play and auto-saves the song at 3.
PF.usable	871	Can it save ahead of time (Android with Jellyfin)?
PF.isUrl	872	Is this URL from the ahead-of-time cache?
PF.url	873	Local URL of a song already saved ahead of time (or null).
PF.soon	874	Schedules an update of the ahead-of-time window.
PF.sync	875	Computes which queue songs to save (N minutes) and asks Android.
fmtMB	890	Formats bytes as MB or GB according to the language.
dlStatus	891	Text of a download's state.
dlStatusShort	902	Short download state for the player button.
dlSummary	908	Downloads summary (count, space, pending).
themeSummary	932	HTML of the chosen theme in Settings.
applyTheme	936	Applies a theme (colors, system bar, bears).

State and routing from line 947 · 8

Function	Line	What it does
regList	953	Registers a song list so buttons can find it.
cur	962	Current screen in the navigation stack.
go	963	Navigates to a screen (root or on top of the current one).
back	972	Goes back: closes dialogs, the player or the current screen.
cacheDrop	982	Forgets cached views starting with a prefix.
refreshIf	984	Redraws the current screen if it meets a condition.
playlistsChanged	987	Forgets and re-requests a playlist that changed.
cached	994	View cache with a cap (oldest go first).

Albums from tags from line 997 · 15

Function	Line	What it does
albumNorm	1003	Normalizes a name to compare albums.
albumKeyOf	1005	Album key of a song from its tags (artist + album + disc).
albumGid	1012	Stable id of a tag-built album.
LIB.who	1015	Identifies the index's server and user.
LIB.index	1017	Tag-based album index (from the encrypted cache or the server).
LIB.peek	1030	The index if it's ready, without waiting.
LIB.reset	1031	Marks the index as stale so it's rebuilt.
LIB.refresh	1032	Rebuilds the index with every song.
LIB.fetchAll	1038	Downloads every song in the library (3 pages at a time).
LIB.pack	1048	Compact version of a song to save the index.
LIB.unpack	1049	Reads a compact song from the saved index, cleaning it.
LIB.save	1054	Saves the encrypted index (if it fits).
LIB.build	1059	Groups songs into albums, without duplicates.
LIB.albumsOf	1084	Albums (by tags) of a group of songs.
LIB.sorted	1089	Sorted album list.

Lyrics search from line 1102 · 10

Function	Line	What it does
lyrNorm	1108	Normalizes lyrics text for searching (no accents or punctuation).
LYRX.load	1111	Loads the saved lyrics index.
LYRX.add	1120	Adds a song's lyrics to the index.
LYRX.save	1121	Saves the encrypted lyrics index (debounced).

Function	Line	What it does
`LYRX.reset`	1129	Deletes the lyrics index.
`LYRX.build`	1131	Builds or completes the index reading lyrics 3 at a time.
`LYRX.search`	1157	Finds songs whose lyrics contain a text.
`lyrSnippetHTML`	1174	Lyrics line with the search highlighted (escaped).
`lyricsHitsHTML`	1179	HTML list of songs found by lyrics.
`lyricsSection`	1184	“In the lyrics” section that fills in while indexing.

Rendering helpers from line 1200 · 8

Function	Line	What it does
`coverHTML`	1202	HTML of a cover with its fallback icon.
`head`	1207	Screen header (title, buttons, bear, back).
`albumCard`	1211	An album card.
`artistCard`	1212	An artist card.
`trackRows`	1213	Container of song rows.
`rowsHTML`	1214	Song rows (number, title, duration, menu).
`loading`	1226	Loading indicator.
`errBox`	1227	Error box with “Retry”.

Views from line 1230 · 6

Function	Line	What it does
`V.albums`	1233	Albums screen (by tags, 120 at a time).
`V.album`	1248	An album's screen.
`normTxt`	1279	Normalizes text (no accents, lowercase).
`mixDef`	1280	A mix's definition (query by genres or years).
`mixTracks`	1294	A mix's songs (cached).
`greeting`	1295	Greeting by time of day.

DJ Nanuq from line 1297 · 82

Function	Line	What it does
`djBearSVG`	1303	SVG of DJ Nanuq with headphones, scarf and turntable.
`djMoment`	1355	Time of day and mood for the “For right now” round.
`pick`	1356	Picks one at random.
`wpick`	1358	Picks at random with weights.
`wshuffle`	1360	Shuffles giving more chance to heavier items.
`djQid`	1363	Queue id carrying the DJ session and round.

Function	Line	What it does
djTag	1364	Reads session and round from a DJ queue id.
djSame	1365	Are two ids from the same DJ session?
DJ.voiceOk	1368	Is there a system voice in the app's language?
DJ.save	1369	Saves the DJ session and shares it with your devices.
DJ.restore	1374	Restores the DJ session when the app opens.
DJ.sample	1381	Random sample of the library to choose genres and decades.
DJ.heavy	1387	Your 60 most played songs.
DJ.pickGenre	1393	Picks a round's genre (avoiding recent ones).
DJ.pickDecade	1403	Picks a round's decade.
DJ.rank	1415	How worn out a song is (lower = goes first).
DJ.pool	1424	Candidate songs for a round type.
DJ.build	1457	Builds a round of ~5 songs with its introduction.
DJ.next	1484	Picks and builds the next round (weights and cooldown).
DJ.items	1498	Turns a round into queue songs and records it.
DJ.lastKey	1506	Number of the last round.
DJ.start	1507	Starts a DJ session (here or on the device you control).
DJ.share	1533	Leaves the DJ state in Jellyfin for another device.
DJ.handedOff	1538	The session moved to another device: it turns off here.
DJ.adopt	1540	Continues here a session coming from another device.
DJ.queueFromShare	1545	Rebuilds the DJ queue from the shared state.
DJ.queueFromTags	1547	Rebuilds the DJ queue from the queue ids.
DJ.remoteTag	1549	Is what's playing on the device you control the DJ?
DJ.shown	1550	Should the DJ be shown?
DJ.remoteTrack	1552	Fetches the rounds of the DJ playing on another device.
DJ.stop	1562	Turns the DJ off (the queue stays as is).
DJ.onTrack	1564	On song change: introduces the new round and adds the next.
DJ.extend	1572	Adds the next round to the end of the queue.
DJ.switchUp	1582	“Change the vibe”: another round instead of what was left.
DJ.announce	1590	Introduces a round (bubble, notice and voice).
DJ.show	1592	Shows the introduction where appropriate.
DJ.speak	1597	Reads the introduction aloud and lowers the music.
DJ.visible	1615	Is the page visible?

Function	Line	What it does
DJ.bears	1616	Turns the DJs' dancing on or off.
DJ.say	1620	DJ speech bubble in the player.
DJ.scene	1625	Record scene in the player (sleeve, vinyl, entries and exits).
DJ.paint	1644	Updates everything DJ-related on screen.
DJSHARE.ok	1674	Can the DJ be shared (Jellyfin and sync on)?
DJSHARE.state	1675	DJ state to share (bounded).
DJSHARE.push	1680	Saves the DJ state in /DisplayPreferences.
DJSHARE.pushRaw	1688	Saves an already built state.
DJSHARE.pushSoon	1692	Saves the state in 1.5 s.
DJSHARE.offSoon	1694	Announces the DJ turned off, if the session is still this device's.
DJSHARE.pull	1698	Reads the shared DJ state.
djClean	1704	Cleans the read DJ state (untrusted data).
djLiveSync	1719	Animates the DJ only while it's visible.
djTileHTML	1731	DJ Nanuq card on Home.
V.dj	1734	DJ Nanuq screen.
nanuqSceneSVG	1758	Nanuq scene for an empty Home cell.
nanuqBear	1774	Nanuq ready to place in a scene.
nanuqWideSVG	1778	Wide Nanuq scene (two cells).
nanuqWideTileHTML	1795	Wide cell with a scene.
nanuqTileHTML	1800	Cell with a scene.
nanuqVisit	1807	A Nanuq from another theme visits a cell.
nanuqVisits	1827	Schedules visits every now and then.
fillGaps	1835	Fills Home's grid gaps with scenes.
tileHTML	1848	A mix tile.
offlineBanner	1853	“No connection” notice with access to Downloads.
V.home	1858	Home screen.
V.mix	1903	A mix's screen.
V.songs	1913	Songs screen (paginated).
V.artists	1941	Artists screen.
fmtLong	1948	Long duration (“3 h 12 min”).
V.artist	1950	An artist's screen (songs, albums, appears on).
V.genres	2030	Genres screen.
V.genre	2035	A genre's screen.

Function	Line	What it does
`V.playlists`	2046	Playlists screen.
`V.playlist`	2051	A playlist's screen.
`V.favorites`	2060	Favorites screen (confirmed with Jellyfin).
`V.folders`	2086	Folders screen.
`V.search`	2095	Search screen (includes lyrics search).
`songsBlock`	2125	Songs block as a list or grid.
`V.queue`	2132	Queue tab (phone).
`V.more`	2133	More screen.
`V.settings`	2136	Settings screen.
`V.downloads`	2199	Downloads screen.
`renderDownloadsSoon`	2215	Redraws Downloads shortly.

Your year and Your week (video) from line 2221 · 37

Function	Line	What it does
`History.purge`	2229	Deletes local history from earlier versions.
`ServerStats.available`	2239	Is the Playback Reporting plugin present and readable?
`ServerStats.plays`	2247	Plays between two dates from the plugin (day by day).
`Recap.total`	2303	Total video length.
`Recap.setMode`	2304	Switches between year and week recap.
`Recap.plays`	2309	Plays for the period (plugin, basic data or demo).
`Recap.tally`	2326	Groups plays: songs, artists, genres, hours, days, streak.
`Recap.collect`	2350	Year recap data.
`Recap.collectWeek`	2362	Data for last week's recap (Monday to Sunday).
`Recap.jellyfinYear`	2378	Songs played in a period according to Jellyfin's basic data.
`Recap.assets`	2391	Prepares the video's images: bears, DJ, covers and palette.
`Recap.loadImg`	2413	Loads an image with limits (8 MB, reduced) for the canvas.
`Recap.sceneAt`	2427	Which scene plays at a given second of the video.
`Recap.frame`	2428	Draws a video frame (with transitions between scenes).
`Recap.transition`	2437	Transition between two scenes (slide, circle, zoom, wipe).
`Recap.bg`	2447	A scene's background: gradient, beat glow and snowflakes.
`Recap.weather`	2464	Draws the music forecast (sun, storm, snow, rainbow...).
`Recap.bear`	2481	Draws a Nanuq hopping to the beat.
`Recap.text`	2488	Text fitted to width, with a shadow.
`Recap.cover`	2498	Cover with rounded corners.

Function	Line	What it does
`Recap.confetti`	2504	Confetti shower.
`Recap.scene`	2512	Draws a recap scene.
`Recap.music`	2759	Video music generated with Web Audio (with a drumroll).
`Recap.stop`	2788	Stops playing or recording the video.
`Recap.play`	2789	Plays the video on the canvas and, if asked, records it (MediaRecorder).
`Recap.save`	2817	Saves the video (Android in chunks, desktop as a download).
`mp4Finalize`	2843	Writes the real duration into the fragmented MP4.
`recapCloseAudio`	2884	Closes the video's audio only once.
`ease.out`	2885	Animation curve (out).
`ease.inOut`	2885	Animation curve (in-out).
`ease.back`	2885	Animation curve with overshoot.
`clamp01`	2886	Clamps a value between 0 and 1.
`mixHex`	2887	Mixes two hex colors.
`hexA`	2888	Hex color with transparency.
`recapYearOpen`	2890	Is it the first week of December?
`recapTileHTML`	2891	“Your year with Nanuq” card on Home.
`V.recap`	2894	Your week / Your year screen (view and export).

Song requests (SomeDL) from line 2937 · 26

Function	Line	What it does
`__nanuqHttp`	2943	Receives a native SomeDL request's response (Android).
`reqNative`	2944	Is there a native bridge to talk to SomeDL without CORS?
`reqRawHttp`	2945	Request to SomeDL through the native bridge or fetch.
`parseSomedl`	2955	Parses the SomeDL address (port 5000 by default).
`reqNetError`	2964	Clear message when SomeDL doesn't respond.
`REQ.save`	2979	Saves the request list.
`REQ.setCfg`	2980	Saves or clears the SomeDL address.
`REQ.pending`	2981	Requests still on the way.
`REQ.visible`	2982	Is the app visible (to check more often)?
`REQ.http`	2983	Request to the SomeDL API with error handling.
`REQ.probe`	2991	Checks that an address is SomeDL.
`REQ.add`	2997	Sends a song request to SomeDL.
`REQ.schedule`	3009	Schedules the next request check.
`REQ.poll`	3010	Checks the requests' state in SomeDL.

Function	Line	What it does
`REQ.announce`	3039	Announces (and notifies) which requests finished.
`REQ.retry`	3054	Requests a failed song again.
`sysNotify`	3061	System notification (Android or desktop).
`askNotifyPermission`	3067	Asks for notification permission in the browser.
`reqRowHTML`	3070	A SomeDL request row.
`reqListHTML`	3079	Request list.
`renderReqSoon`	3086	Redraws the request list shortly.
`V.request`	3090	Request a song screen.
`refreshFromJellyfin`	3150	“Refresh from Jellyfin”: forgets caches, rebuilds indexes and refreshes the queue.
`sessionGone`	3173	Confirms whether a 401 really is an expired session.
`sessionExpired`	3184	Expired session: stops everything and asks to log in.
`render`	3192	Draws the current screen with a transition and error handling.

`Player` from line 3228 · 53

Function	Line	What it does
`P.pause`	3238	Pauses, marking that Nanuq requested it.
`saveQueue`	3241	Saves the queue and updates the ahead-of-time window.
`QPOS.save`	3246	Saves the song's second to resume it.
`QPOS.restore`	3252	Where to resume the song when the app opens.
`QPOS.clear`	3256	Forgets the saved position.
`setQueue`	3260	Replaces the queue and starts playing.
`addToQueue`	3261	Adds songs at the end or next.
`ptime`	3271	Current second of the song (here or on the other device).
`pdur`	3272	Song duration.
`setSrc`	3278	Changes the audio source resetting its state.
`seekTo`	3282	Seeks to a second (for live conversions it requests from there).
`playIndex`	3303	Plays song N of the queue.
`armStallWatch`	3333	Warns if a song takes too long to start.
`MP3.add`	3347	Remembers a song that must be requested as MP3.
`localFailed`	3351	The local copy failed: it plays from the server (and is downloaded again).
`canFallback`	3363	Can this song be requested as MP3?

Function	Line	What it does
`fallbackMp3`	3366	Requests the song as MP3 320 kbps if the original won't start.
`armFallback`	3376	Schedules the switch to MP3 if it doesn't start in 6 s.
`playFailed`	3382	Explains why it couldn't play.
`audioDead`	3403	Is the audio dead (network error)?
`replayHere`	3404	Requests the song again from the second it was at.
`fadeStop`	3412	Stops a fade in progress.
`fadeTo`	3413	Fades the volume to a value.
`userVol`	3425	The volume you chose (lowered if the DJ is talking).
`pauseSoft`	3426	Pauses with a fade.
`playSoft`	3431	Resumes with a fade (or reloads if another app paused).
`partyNow`	3440	Do the controls handle the party right now?
`partyHostOpen`	3443	Is a party room open on this computer?
`toggle`	3444	Play/pause (party, remote control or here).
`next`	3453	Next song (respects repeat and the end of the queue).
`prev`	3465	Previous song (or restart if past 3 s).
`seekFrac`	3466	Seeks to a fraction of the song.
`removeAt`	3467	Removes a song from the queue.
`clearQueue`	3472	Clears the queue.
`shuffleQueue`	3473	Shuffles the queue (the current one stays first).
`setMode`	3481	Changes the repeat mode.
`updateModeBtn`	3485	Updates the repeat button.
`loadPeaks`	3487	Loads a song's waveform.
`cssVar`	3496	Reads a theme CSS variable (cached).
`drawWaves`	3497	Draws the progress bars or the waveform.
`roundRect`	3525	Rounded rectangle on canvas.
`updateNow`	3527	Updates everything that shows the current song.
`updatePlayState`	3557	Play/pause icons and system controls state.
`DISC.can`	3572	Is there a Discord bridge (desktop)?
`DISC.update`	3573	Schedules the update to Discord.
`DISC.send`	3574	Sends Discord what's playing (or clears it).
`dcStatusPaint`	3590	Discord status line in Settings.
`rcKeep`	3611	Silent audio so the system keeps the controls.

Function	Line	What it does
sessionHold	3625	Keeps the system controls while loading or when the queue ends.
rcKeepOff	3631	Stops the silent audio.
stallCheck	3671	Detects audio stuck mid-song and requests it again.
netBack	3680	Network is back: resumes what was cut and reconnects.
paintVol	3711	Draws the volume bar and its icon.

Lyrics from line 3716 · 32

Function	Line	What it does
lrcKey	3719	Key of lyrics saved on the device.
lrcTime	3720	Formats a time as mm:ss.cc (LRC).
parseLRC	3721	Reads an LRC file (timestamps and offset).
toLRC	3735	Builds a clean LRC file.
cleanLyricsText	3746	Cleans typed or pasted lyrics (control chars, HTML, size).
lyricsFromText	3755	Converts text (LRC or plain) into lyrics.
localLyrics	3756	Lyrics saved on this device.
lyrSame	3764	Are two lyrics the same (text and timing)?
lyricsSaved	3772	Shows freshly saved lyrics until Jellyfin serves them.
getLyrics	3787	A song's lyrics (local, pending, cache or server).
setLyricsMode	3795	Opens or closes the lyrics with a transition.
setQueueMode	3808	Opens or closes the player's queue with a transition.
renderSheetQueue	3819	Queue list inside the player.
applyLyricsMode	3828	Applies lyrics mode without a transition.
reloadLyrics	3836	Reloads the current lyrics.
renderLyrics	3837	Draws the current song's lyrics.
estimateTimes	3861	Spreads approximate timing for unsynced lyrics.
syncLyrics	3868	Highlights the playing line and scrolls the lyrics.
refreshLyricsButtons	3904	Lyrics or Find lyrics button depending on whether there are lyrics.
syncStart	3919	Opens the editor to sync by hand.
syncEnd	3933	Closes the editor.
syncCancel	3937	Cancels syncing.
renderSync	3938	Draws the sync editor.
syncEditorTick	3945	Marks the playing line in the editor.
syncMark	3952	Marks the start of the highlighted line.

Function	Line	What it does
syncUndo	3965	Undoes the last mark.
syncNudge	3973	Moves the last mark ¼ s.
syncSave	3982	Fills in timing and saves the LRC.
saveLyricsText	3999	Saves lyrics in Jellyfin (or on the device if it can't).
lyricsSearch	4011	Searches LrcLib for lyrics and lets you choose one.
lyricsPaste	4041	Dialog to paste or edit lyrics.
forgetLocalLyrics	4059	Removes the lyrics saved on this device.

Queue from line 4062 · 4

Function	Line	What it does
renderQueue	4064	Draws the queue panel.
renderQueueAll	4073	Redraws every queue view.
moveQueue	4078	Moves a song in the queue (also in remote control).
markCurrent	4130	Highlights the playing song in lists.

Toasts, menus and dialogs from line 4133 · 2

Function	Line	What it does
toast	4135	Brief on-screen notice.
menu	4136	Options menu (bottom sheet).

My devices from line 4144 · 63

Function	Line	What it does
devLabel	4149	Friendly name of a device.
devName	4151	Device name for sentences.
expand6	4153	Expands an IPv6 address.
ipKey	4159	Network and privacy of an IP address.
sameNetwork	4167	Are two addresses on the same network?
RC.on	4179	Is My devices turned on?
RC.start	4180	Starts My devices (WebSocket and polling).
RC.stop	4181	Stops My devices.
RC.connect	4187	Opens Jellyfin's WebSocket with retries.
RC.send	4200	Sends a message over the WebSocket.
RC.onMessage	4201	Handles WebSocket messages (commands, favorites, keep-alive).
RC.fromMe	4212	Does the command come from your own user?
RC.onPlaystate	4219	Runs received pause, next, seek...

Function	Line	What it does
RC.onPlay	4232	Runs a play command (and adopts the DJ if it comes with it).
RC.onGeneral	4254	Runs received repeat or shuffle.
RC.queueForReport	4261	Queue reported to Jellyfin.
RC.reportSoon	4262	Reports the state to Jellyfin shortly.
RC.schedule	4268	Schedules the next device poll.
RC.poll	4269	Looks for your other Nanuqs and follows the one you control.
RC.maybeFollow	4292	Connects automatically to another Nanuq playing on your network.
RC.enter	4298	Starts controlling another device.
RC.leave	4314	Stops controlling and restores your queue.
RC.apply	4322	Mirrors here what's playing on the other device.
RC.pos	4353	Estimated second of the song on the other device.
RC.tick	4354	Updates bar, lyrics and system controls in remote control.
RC.decorate	4361	Shows “Playing on …” in the player.
RC.fail	4367	Warns that a command couldn't be sent.
RC.cmd	4368	Sends a command to the playing device.
RC.seek	4374	Seeks on the other device.
RC.window	4375	Queue window that is sent (≤ 200).
RC.playQueue	4376	Sends a queue to play there.
RC.add	4385	Adds songs to the queue there.
RC.reorder	4395	Sends the reordered queue (keeping the DJ).
RC.general	4406	Sends a general command there.
RC.here	4408	“Play here”: brings the music (and the DJ) to this device.
RC.there	4431	Sends the music (and the DJ) to another device.
__nanuqTick	4450	20-second heartbeat from Android (WebSocket, party, stuck audio).
__nanuqVis	4458	Android reports whether the app is visible.
openDevices	4459	My devices menu.
updateFavBtn	4490	Player's favorite button.
setFavLocal	4496	Marks the favorite in the local queue.
favPending	4503	Remembers a recent favorite change.
favMerge	4504	Applies recent changes over the server's list.
favHold	4522	Protects the playing song's favorite (Jellyfin 12 bug).
favHeld	4525	Is this favorite being protected?

Function	Line	What it does
favReassert	4526	Saves the favorite again after each playback report.
favRemote	4534	Handles favorite changes made elsewhere.
refreshFav	4556	Confirms with the server whether the song is a favorite.
toggleFav	4567	Marks or unmarks the favorite and verifies it.
favConfirm	4587	Second confirmation against the favorites section.
updateDlBtn	4601	Player's Download button.
dlAddToast	4610	Notice when requesting a download.
dlTap	4619	Tap on Download (request or manage).
connTest	4632	“Test connection”: measures API, cover, audio, seeking and player.
startRadio	4760	Radio from a song.
addToPlaylistMenu	4774	Menu to add to a playlist (or create one).
songOptions	4789	Common song options.
queueMenu	4796	Menu for a queue song.
trackMenu	4809	Menu for a song in a list.
askConfirm	4817	Confirmation dialog.
askName	4823	Dialog to type a name.
openSheet	4831	Opens the full-screen player.
closeSheet	4839	Closes the full-screen player.

Party (watch party) from line 4842 · 65

Function	Line	What it does
partyHost	4849	Can this device host (desktop)?
partyGuest	4850	Is this device a guest (Android)?
partyOn	4851	Is the party available on this device?
partySongUrl	4853	MP3 320 kbps URL to send a song to the party.
partyCover	4859	240 px JPEG thumbnail of the cover.
partyCoverOk	4869	Does the cover come from the room (127.0.0.1 with token)?
partyClean	4872	Cleans the state sent by the computer (untrusted).
stopForParty	4884	Stops your music when opening the room.
PH.active	4899	Does the party handle the controls?
PH.init	4900	Prepares the party player and listens to main.js.
PH.onEv	4912	Handles room events (state, joined, left).
PH.apply	4918	Applies the room state (computer party mode).
PH.list	4930	Room song list.

Function	Line	What it does
PH.open	4931	Opens the room.
PH.close	4937	Closes the room.
PH.jump	4939	Plays a song from “Up next” right now.
PH.order	4945	Changes the list order.
PH.next	4947	Follows the list: plays the next ready song.
PH.play	4954	Plays a party song.
PH.stopCur	4962	Stops the current party song.
PH.done	4970	A song ended: deletes it and moves on.
PH.toggle	4971	Pauses or resumes the party.
PH.yieldTo	4984	You played your music: the party pauses and hands over the controls.
PH.media	4991	System controls with the party song.
PH.report	5003	Tells the room what's playing.
PH.loadLyrics	5008	Loads the party song's lyrics.
PH.line	5018	Current and next lyrics line.
PH.tick	5026	Karaoke: every second, time and line to the phones.
PG.connect	5036	Joins a room (with code or key) and reconnects on its own.
PG.leave	5063	Leaves the party.
PG.joined	5073	Are you at a party?
PG.ui	5074	Phone party mode (hides bars and player).
PG.wake	5083	The phone woke up: rejoins if needed.
PG.resume	5089	When opening the app, returns to the ongoing party.
PG.abortUp	5094	Cancels the upload in progress.
PG.onMsg	5095	Handles room messages.
PG.add	5122	Sends a song to the party (lyrics, cover and audio in chunks).
PG.remove	5178	Cancels a song that's still arriving.
PG.find	5180	Finds rooms on the network (UDP from Android).
partyAddr	5200	Converts the typed address into ws://.../fiesta.
iglooHTML	5211	Igloo HTML.
iglooBear	5227	A guest's Nanuq in their theme's costume.
iglooHoldSVG	5234	Cover held up with sparkles (who picked the song).
iglooHeldOnTop	5245	Puts the object the Nanuq was holding on top of the cover.
partyGuestColor	5256	Label color from the guest's theme.
partyCoverSrc	5264	Cover URL of a party song.

Function	Line	What it does
iglooSync	5272	Places guests in the circle without redrawing.
iglooLive	5306	Animates the igloo only while visible.
iglooQueue	5311	“Up next” row in full screen.
iglooQueueEdge	5323	Gradient if there are more songs out of view.
partyVolumeStep	5399	Volume with ↑ ↓ in full screen and its notice.
iglooIdle	5412	Hides the pointer after 2.5 s idle.
iglooVinyl	5422	Sleeve and vinyl record of the playing song.
iglooLine	5434	Lyrics on the ceiling with a sparkle.
renderPartySoon	5441	Redraws the party shortly.
partyPaintTick	5445	Updates the karaoke (line, next, time).
partyPaintUp	5451	Progress of the song you're sending.
partyRowsHTML	5452	Party song list.
partyGuestsHTML	5461	Guest list.
partyNowHTML	5462	What's playing at the party.
paintParty	5467	Draws the party screen (host or guest).
partySongsHTML	5528	Songs to send.
partySearch	5529	Party search (playlists, artists, albums, songs, lyrics).
partyBrowse	5557	Songs of an artist, album or playlist in the party.
V.party	5572	Party screen.

Global actions from line 5586 · 1

Function	Line	What it does
d0	5588	Shortcut for an element's dataset.

Login from line 5746 · 11

Function	Line	What it does
recentServers	5749	Servers used before.
rememberServer	5750	Remembers a server.
forgetServer	5756	Forgets a server.
probeTimed	5758	Tests a server measuring how long it takes.
canDiscover	5769	Can servers be discovered on the network?
discoverServers	5771	Discovers Jellyfin on the network (UDP 7359).
serverList	5790	Server list with their status.
cleartextRisk	5817	Would the password travel unencrypted over the internet?
showLogin	5828	Login screen.

Function	Line	What it does
`startApp`	5894	Starts the app with the session (queue, position, DJ, Home).
`logout`	5910	Logs out and deletes personal data.

17 Rest of the page

Languages and service worker.



web/i18n.js

1,004 lines · 1 functions

Function	Line	What it does
tx	995	Translates a Spanish text into the chosen language and fills in the {0}, {1}... values.

web/sw.js

45 lines · 3 functions

Function	Line	What it does
install	8	Caches the app's files when the service worker installs.
activate	12	Deletes caches from older versions.
fetch	16	Serves only the app's files (network first for HTML/JS/CSS); the API and music go straight to the network.

18 Android

Native Java and the JavaScript bridge.



android/app/src/main/java/com/nanuq/app/Discovery.java

78 lines · 3 functions

Function	Line	What it does
`Discovery.Discovery`	27	Private constructor (static methods only).
`Discovery.find`	30	Discovers over UDP (Jellyfin or parties) and returns the answers as JSON.
`Discovery.send`	72	Sends the question to every broadcast address.

android/app/src/main/java/com/nanuq/app/Downloads.java

591 lines · 44 functions

Function	Line	What it does
`Downloads.get`	43	Single instance of downloads.
`Downloads.Downloads`	72	Sets up folder, index, local server and network watch.
`Downloads.audio`	83	Audio file for the local server.
`Downloads.audioMime`	84	Audio type for the local server.
`Downloads.cover`	85	Cover for the local server.
`Downloads.prefetch`	86	Ahead-of-time song for the local server.
`Downloads.prefetchMime`	87	Type of the ahead-of-time song.
`Downloads.onAvailable`	91	Network is back: retries downloads.
`Downloads.onCapabilitiesChanged`	92	Network changed: checks the download queue.
`Downloads.setListener`	97	Connects the page to receive changes.
`Downloads.configure`	101	Maximum songs and Wi-Fi only.
`Downloads.base`	117	Local server address.
`Downloads.okUrl`	127	Is the URL shaped like a Jellyfin audio or cover?
`Downloads.openSameHost`	141	Opens a connection following redirects only to the same server.
`Downloads.enqueue`	162	Adds a download (or updates it).
`Downloads.setScore`	196	Updates how many times a download played.
`Downloads.remove`	202	Removes a download.
`Downloads.removeAll`	208	Removes all (or only automatic ones).
`Downloads.list`	214	List of downloads as JSON.

Function	Line	What it does
`Downloads.network`	222	Current network (wifi, cellular, none), looking beneath a VPN.
`Downloads.underlying`	236	Real network beneath a VPN.
`Downloads.file`	245	File of a ready download.
`Downloads.coverFile`	253	A download's cover.
`Downloads.setCover`	262	Requests an old download's cover.
`Downloads.fetchCoverLater`	270	Downloads the cover in the background.
`Downloads.fetchCover`	273	Downloads the cover (≤ 15 MB).
`Downloads.mime`	301	A download's audio type.
`Downloads.retryAndKick`	309	Retries failed ones and continues.
`Downloads.kick`	316	Starts the download queue.
`Downloads.kickLocked`	318	Starts the queue (already locked).
`Downloads.loop`	324	Downloads pending ones one at a time.
`Downloads.nextPending`	344	The next one (yours first).
`Downloads.download`	354	Downloads a song with limits and free space.
`Downloads.victim`	427	The least-played automatic one (to make room).
`Downloads.removeEntry`	436	Deletes a download and its files.
`Downloads.caps`	450	Active network capabilities.
`Downloads.unmetered`	455	Is the network unmetered (Wi-Fi, Ethernet)?
`Downloads.networkAllowed`	460	Can it download on this network?
`Downloads.safe`	468	Safe file name for an id.
`Downloads.toJson`	470	Download as JSON for the page.
`Downloads.emit`	488	Notifies the page of a change.
`Downloads.ensureLoaded`	494	Reads the index when the store opens.
`Downloads.load`	496	Reads the encrypted index (or recovers from files).
`Downloads.save`	562	Saves the encrypted index.

`android/app/src/main/java/com/nanuq/app/LocalServer.java` 184 lines · 7 functions

Function	Line	What it does
`LocalServer.LocalServer`	46	Opens the server on 127.0.0.1 with a random token.
`LocalServer.base`	60	Base address with the token.
`LocalServer.acceptLoop`	62	Accepts connections (max. 32 at a time).
`LocalServer.readLine`	74	Reads a request line with a cap and a deadline.
`LocalServer.handle`	86	Handles a request (10 s and ≤ 100 headers).
`LocalServer.status`	129	Answers a status code.

Function	Line	What it does
`LocalServer.serveFile`	135	Serves a file with ranges (seeking within the song).

android/app/src/main/java/com/nanuq/app/MainActivity.java

406 lines · 44 functions

Function	Line	What it does
`MainActivity.run`	63	Heartbeat every 20 s to the page.
`MainActivity.onCreate`	70	Creates the WebView, connects services and loads the app.
`MainActivity.createWeb`	88	Configures the WebView (no file access, bridge).
`MainActivity.recreateWeb`	114	Recreates the WebView if the system killed it.
`MainActivity.dispatchMedia`	124	Passes a notification button to the page.
`MainActivity.onBackPressed`	135	Back: the page decides or the app goes to the background.
`MainActivity.onStart`	144	The app came back into view.
`MainActivity.onStop`	147	The app went to the background (saves the store).
`MainActivity.onDestroy`	150	Releases the WebView and the notification.
`MainActivity.setBarColor`	164	System bar color according to the theme.
`MainActivity.showNotice`	185	“Requested song ready” notification.
`MainActivity.setMetadata`	213	Bridge: title, artist, album and cover.
`MainActivity.setPlaybackState`	214	Bridge: playing or paused.
`MainActivity.setActions`	215	Bridge: available buttons.
`MainActivity.setPosition`	216	Bridge: position and duration.
`MainActivity.dlList`	224	Bridge: download list.
`MainActivity.dlEnqueue`	225	Bridge: download a song.
`MainActivity.dlCover`	226	Bridge: request a cover.
`MainActivity.dlBase`	227	Bridge: local server address.
`MainActivity.dlRemove`	228	Bridge: remove a download.
`MainActivity.dlRemoveAll`	229	Bridge: remove downloads.
`MainActivity.dlConfig`	230	Bridge: download settings.
`MainActivity.dlScore`	231	Bridge: plays of a download.
`MainActivity.dlNetwork`	232	Bridge: network type.
`MainActivity.pfSet`	235	Bridge: ahead-of-time window.
`MainActivity.pfReady`	236	Bridge: is that song already saved?
`MainActivity.pfBroken`	237	Bridge: that saved song didn't play.
`MainActivity.http`	240	Bridge: CORS-free request to SomeDL.
`MainActivity.secKind`	248	Bridge: encrypted store state.
`MainActivity.secGet`	249	Bridge: read an encrypted value.

Function	Line	What it does
`MainActivity.secSet`	250	Bridge: save an encrypted value.
`MainActivity.secDel`	251	Bridge: delete an encrypted value.
`MainActivity.secReset`	253	Bridge: start over.
`MainActivity.videoBegin`	257	Bridge: starts saving a video (MP4/WebM).
`MainActivity.videoChunk`	269	Bridge: a chunk of the video (≤ 300 MB).
`MainActivity.videoEnd`	281	Bridge: finishes and saves in Movies › Nanuq.
`MainActivity.videoShare`	323	Bridge: share the video just saved.
`MainActivity.notice`	332	Bridge: system notification.
`MainActivity.partyDiscover`	335	Bridge: find parties over UDP.
`MainActivity.discover`	343	Bridge: find Jellyfin over UDP.
`MainActivity.onRenderProcessGone`	354	The WebView process died: it's recreated.
`MainActivity.shouldInterceptRequest`	360	Serves the app from the assets at http://localhost.
`MainActivity.shouldOverrideUrlLoading`	377	Only http(s) links go out to the browser.
`MainActivity.mime`	399	Type of an asset file.

`android/app/src/main/java/com/nanuq/app/MediaCenter.java` 372 lines · 33 functions

Function	Line	What it does
`MediaCenter.get`	52	Single instance of the media center.
`MediaCenter.MediaCenter`	72	Creates the MediaSession and the notification channel.
`MediaCenter.onPlay`	83	System play button.
`MediaCenter.onPause`	84	System pause button.
`MediaCenter.onStop`	85	System stop button.
`MediaCenter.onSkipToNext`	86	System next button.
`MediaCenter.onSkipToPrevious`	87	System previous button.
`MediaCenter.onSeekTo`	88	Seek from the system.
`MediaCenter.onFastForward`	89	Fast-forward from the system.
`MediaCenter.onRewind`	90	Rewind from the system.
`MediaCenter.attach`	96	Connects the page.
`MediaCenter.detach`	98	Disconnects the page.
`MediaCenter.hasPage`	101	Is there a page to receive commands?
`MediaCenter.stopIfNoPage`	104	Without a page, removes the notification.
`MediaCenter.dispatch`	113	Sends an action to the page.
`MediaCenter.setMetadata`	120	Song details (and loads the cover).
`MediaCenter.setPlaybackState`	141	Playback state.

Function	Line	What it does
`MediaCenter.setActions`	152	Available buttons.
`MediaCenter.setPosition`	160	Position and duration.
`MediaCenter.isPlaying`	174	Is it playing?
`MediaCenter.currentPosition`	176	Estimated current position.
`MediaCenter.publishMetadata`	181	Publishes the details on the MediaSession.
`MediaCenter.publishState`	191	Publishes the state and buttons.
`MediaCenter.refresh`	212	Adjusts service, notification and locks.
`MediaCenter.shutdown`	240	Stops everything when the app closes.
`MediaCenter.buildNotification`	251	Builds the playback notification.
`MediaCenter.action`	280	A notification button.
`MediaCenter.serviceIntent`	284	A button's intent to the service.
`MediaCenter.contentIntent`	292	Open the app from the notification.
`MediaCenter.onReceive`	300	Headphones unplugged: pause.
`MediaCenter.setNoisyReceiver`	305	Listens for headphone disconnection.
`MediaCenter.loadBitmap`	319	Loads the cover with limits (8 MB, reduced).
`MediaCenter.readCapped`	364	Reads with a byte cap.

android/app/src/main/java/com/nanuq/app/NativeHttp.java 84 lines · 5 functions

Function	Line	What it does
`NativeHttp.NativeHttp`	19	Private constructor (static methods only).
`NativeHttp.Result`	24	Response (status and text).
`NativeHttp.allowed`	30	Allowed SomeDL route?
`NativeHttp.fetch`	37	CORS-free request only to SomeDL, no redirects.
`NativeHttp.readAll`	71	Reads the response (≤ 4 MB).

android/app/src/main/java/com/nanuq/app/PlaybackService.java 101 lines · 7 functions

Function	Line	What it does
`PlaybackService.onCreate`	31	Prepares the CPU and Wi-Fi locks.
`PlaybackService.onStartCommand`	46	Foreground and notification buttons.
`PlaybackService.setLocks`	65	Takes or releases the locks.
`PlaybackService.release`	76	Leaves the foreground and ends.
`PlaybackService.onTaskRemoved`	84	Removed from recents: closes everything.
`PlaybackService.onDestroy`	92	Releases the locks.
`PlaybackService.onBind`	99	No binding (started service).

android/app/src/main/java/com/nanuq/app/Prefetch.java 222 lines · 16 functions

Function	Line	What it does
Prefetch.get	40	Single instance of the ahead-of-time cache.
Prefetch.Prefetch	61	Empties the previous cache and watches the network.
Prefetch.onAvailable	71	Network is back: keeps downloading.
Prefetch.set	80	Receives the desired window and deletes what's left over.
Prefetch.broken	109	A saved one didn't play: deleted and not repeated.
Prefetch.ready	116	Is that song ready?
Prefetch.file	118	File of a ready song.
Prefetch.mime	125	Audio type.
Prefetch.drop	130	Removes a song and its file.
Prefetch.deleteFiles	132	Deletes a song's files.
Prefetch.kick	139	Starts the downloads.
Prefetch.kickLocked	141	Starts (already locked).
Prefetch.loop	147	Downloads pending ones one at a time (respects the total cap).
Prefetch.online	160	Is there a network?
Prefetch.fetch	166	Downloads a song with limits.
Prefetch.trim	211	If over the total cap, drops the farthest ones.

android/app/src/main/java/com/nanuq/app/SecureStore.java 192 lines · 16 functions

Function	Line	What it does
SecureStore.get	44	Single instance (or reads a value if given a key).
SecureStore.SecureStore	58	Opens the store when created.
SecureStore.open	69	Opens key (Keystore) and file: ready, locked or lost.
SecureStore.kind	104	State (retries if it was locked).
SecureStore.reset	113	Start over: sets the file aside and creates a new key.
SecureStore.ready	122	Is it open?
SecureStore.set	129	Saves a value (≤ 8 MB).
SecureStore.del	136	Deletes a value.
SecureStore.validKey	142	Valid key?
SecureStore.scheduleSave	144	Saves in 400 ms.
SecureStore.flush	150	Saves now.
SecureStore.saveNow	152	Encrypts and writes the file.
SecureStore.encrypt	161	Encrypts with AES-256-GCM.
SecureStore.decrypt	171	Decrypts and verifies.

Function	Line	What it does
`SecureStore.readAll`	178	Reads a file (≤ 64 MB).
`SecureStore.writeAtomic`	186	Writes atomically.

android/bridge/android-bridge.js 97 lines · 9 functions

Function	Line	What it does
`abs`	17	Turns a relative URL into an absolute one.
`MediaMetadata`	19	MediaMetadata replacement for the page.
`bestArt`	28	Chooses the best cover (≤ 512 px) for the notification.
`mediaSession.metadata`	42	Sends title, artist, album and cover to Android.
`mediaSession.playbackState`	47	Sends the playback state to Android.
`mediaSession.setActionHandler`	54	Registers the buttons the page handles.
`mediaSession.setPositionState`	58	Sends the position only when it drifts.
`__nanuqMedia`	76	Runs a notification or headphone button on the page.
`sendTheme`	87	Sends the theme color to the system bars.

android/build-apk.sh 19 lines · 1 functions

Function	Line	What it does
`(script)`	1	Builds the APK and leaves it in dist/<version>/.

19 Desktop

Electron and its packaging.



linux/main.js

855 lines · 61 functions

Function	Line	What it does
serve	47	Hardened internal 127.0.0.1:47863 server that hands the app to the window (and the party).
broadcastOf	79	Broadcast address of an IPv4 network.
discover	80	Discovers Jellyfin servers over UDP 7359.
fromApp	100	Does the IPC message come from the app's page?
vaultBackend	132	Which system keyring is available (GNOME Keyring, KWallet, DPAPI, Keychain).
secFile	148	Path of the encrypted data file.
secKeyFile	149	Path of the data key encrypted with the keyring.
writeAtomic	150	Writes a file atomically (0600).
secInit	153	Opens the encrypted store (AES-256-GCM): locked, lost or ready.
secFlush	190	Writes the encrypted store to disk.
dcSet	228	Records the Discord connection state.
dcPaths	229	Possible Discord sockets or pipes (only yours).
dcWrite	240	Sends a message in Discord's format.
dcRetry	241	Looks for Discord again in 10 s.
dcConnect	242	Connects to Discord trying each socket and greets it.
dcData	258	Reads Discord's messages with limits (READY, ERROR, close, ping).
dcFlush	281	Sends the activity respecting Discord's rate (and the minimal one if rejected).
dcActivity	292	Builds the clean, bounded “Listening” activity.
jparse	329	Parses outside JSON without toString/valueOf/__proto__.
ptStr	330	Converts to safe, bounded text (no control chars).
ptIp	331	IP address without the ::ffff: prefix.
ptLocal	335	Is the address on your local network?
ptOpenNow	342	Is the room open?

Function	Line	What it does
ptEmit	343	Notifies the page of a room event.
ptBase	348	Folder on disk where the room's songs are stored (on Linux, ~/.cache/nanuq; it used to be the temp folder, which is RAM in the Flatpak).
ptCleanTemp	353	Deletes temporary folders from earlier rooms.
ptNonce	366	AES-GCM nonce for a block: the song's 8 random bytes + the block number.
ptSeal	367	Encrypts and authenticates a 64 KB block of a party song with the room key.
ptEncSize	372	Exact size the encrypted file must have (detects truncated or extended files).
ptOpenRange	373	Reads, checks and decrypts only the blocks of a requested range; fails if a block was altered.
ptSniff	389	Is it audio? (by its first bytes).
ptCover	402	Validates the cover a phone sends (JPEG/PNG/WebP ≤ 150 KB).
ptPublic	411	Public data of a song in the list.
ptGuests	415	Present and away guests.
ptAddrs	416	This computer's IPv4 addresses.
ptStatus	421	Full room state for the page.
ptState	427	Broadcasts the state to everyone (batched in 150 ms).
ptStateNow	428	Sends the state now.
ptKill	434	Removes a song from the list and the disk.
ptFrame	443	Writes a WebSocket frame.
ptSend	450	Sends a JSON message to a phone.
ptDrop	451	Closes a phone's connection (with a reason).
ptGone	459	A phone disconnected (stays away or left).
ptUpgrade	470	Accepts the WebSocket connection (local network only, limits).
ptData	490	Reads WebSocket frames with every validation.
ptMsg	523	Handles a message from a phone.
ptNewCode	536	Changes the room code.
ptHello	540	A phone joining: code, key, brute force.
ptAdd	570	A phone announces a song (with limits).
ptChunk	588	Receives a piece of the song and writes it to disk.
ptEnd	606	The song finished arriving: checks it's audio.

Function	Line	What it does
ptServe	623	Serves the room's songs and covers to the window (with Range).
ptGuestCover	651	Serves covers to the phones (only with their key).
ptGc	660	Cleanup every 30 s (old, half-arrived, away, silent).
ptListen	670	Opens the room port (47864 or a free one).
ptOpen	683	Opens the room: server, UDP, code and token.
ptClose	710	Closes the room and cuts every connection.
stateFile	770	File with the window's size and position.
loadState	772	Reads the window's size and position.
saveState	777	Saves the window's size and position.
start	783	Starts the app: store, server, window, permissions and protections.

linux/preload.js

25 lines · 6 functions

Function	Line	What it does
nanuqDesktop.discover	7	Bridge: discover Jellyfin on the network.
nanuqDesktop.http	8	Bridge: CORS-free request to SomeDL.
nanuqDesktop.discord	11	Bridge: Discord activity (or its status).
nanuqDesktop.party.op	14	Bridge: party operations (open, close, jump, order, karaoke...).
nanuqDesktop.party.on	15	Bridge: listen to room events.
nanuqDesktop.secure.*	17	Synchronous bridge to the encrypted store (kind, get, set, del, reset).

linux/prepare-web.js

16 lines · 1 functions

Function	Line	What it does
(script)	3	Copies web/ to www/ and takes the version from app.js.

linux/collect.js

15 lines · 1 functions

Function	Line	What it does
(script)	3	Moves the built packages to dist/<version>/.

linux/flatpak/build-flatpak.sh

54 lines · 1 functions

Function	Line	What it does
(script)	1	Builds the Flatpak, publishes it to the signed local repository and installs or updates it.

20 Build

The script that builds everything.



scripts/build.sh

68 lines · 1 functions

Function	Line	What it does
(script)	1	Bumps the version and builds Android, Linux, Flatpak and Windows.

Made with care (and lots of ice)

If you change something, remember: the interface lives in web/ and every text goes through tx().

